Spark Programming

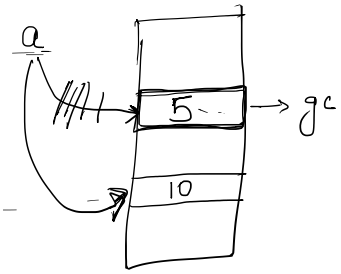
Sunday, 30 August 2026 12:13 AM

* RDD : Resilient Distributed Dataset

* 1) Immutable: Value can't be changed

a = 5

a = 10



rdd-1 → operation → rdd-2.

Spark remembers how RDD was built

2) Resilient

If any operation fails in between, spark knows the lineage.

op5 failed.

op1 → op2 → op3 → - - - - - → op10.

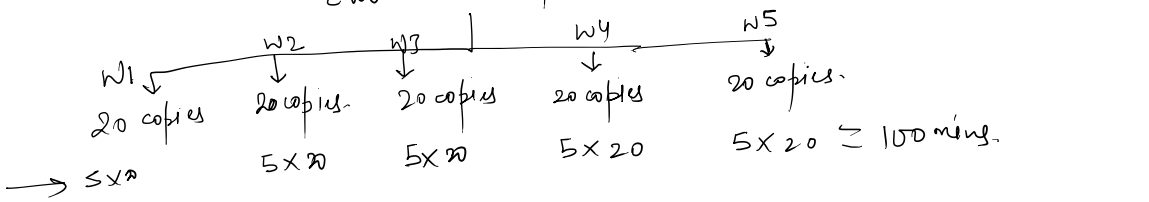
3) Parallelism

Large data processing tasks are broken into independent smaller chunks.

1 copy = 5 mins

Job.

evaluate 100 copies.



* SparkContext

- It is the starting point of any spark application.
- It represents a connection between your application and the spark cluster.
- 1 SparkContext, your program communicates with cluster manager.

- Thorough [illegible] requests resources and coordinated execution.
- It works as a control center.
- Is responsible for tracking RDDs and their lineage.

```
from pyspark import SparkConf, SparkContext
```

- SparkConf → used to define configuration setting for your Spark app
- SparkContext → is the entry point to spark's execution engine.

```
conf = SparkConf() \
  .setAppName("MyFirstSparkApp") \
  .setMaster("local[*]")
```

- SparkConf → creates an empty config object
- setAppName → assigns a human-readable name to your app
- setMaster("local[*]") → tells Spark where to run the app
  - ↳ local – run on local machine
  - ↳ [*] – use all CPU cores

```
sc = SparkContext(conf = conf)
```

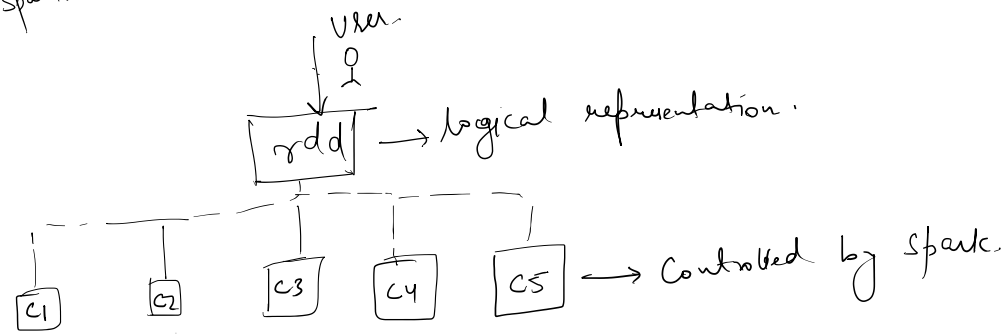
Starts Spark for your application.

SparkContext:
- Connects to the cluster manager
- Requests resources (CPU, memory)
- Initializes internal services (scheduler, task manager)
- It can be used to create RDDs
  - Read files
  - Submit distributed jobs.

[illegible] SparkSession [illegible] for working with

* SparkSession:

- It is the modern and recommended entry point for [illegible] Spark.
- It provides a unified, single interface for interacting with all Spark functionalities, → RDDs, Dataframes, Datasets, SQL and Hive.
- It acts like a wrapper and coordinator and internally manages SparkContext and also initializes high level components such as Spark SQL.



* Lambda Function:

Anonymous or Ghost Function

Lambda args : expression. → Lambda x : x**2 (x^2).

Keyword ↑ Lambda, ↑ arg x, ↑ exp x**2

* Narrow vs Wide Transformation.

Aspect	Narrow Transformation	Wide Transformation
Definition	Output depends on one Partition	Output depends on multiple partitions
Data Movement	Within Partition	Across partitions
Shuffle	No shuffle	Needs shuffle between partitions
	Faster	Slower

Speed		
Fault Recovery	Simple Recovery : only within the partitions	Complex Recovery.
Resource Usage	Low CPU, disk and network	High
Examples	map, filter, flatMap, mapPartitions, Union.	reducebykey, groupbykey, join, distinct, cogroup.

* Actions.

- Trigger the execution of all preceding transformations.
- Returns a result to the driver or writes data
- Marks the end of a Spark computation pipeline.

Common Spark Actions :

Action	Description	Example -
count()	Returns the total number of elements in RDD	rdd.count()
collect()	Execute the computation and brings all elements to the driver as a list	rdd.collect()
take(n)	Execute the computation and returns first n elements from RDD	rdd.take(5)
first()	Execute the computation and returns first element from RDD	rdd.first()
reduce()	Aggregate all elements of a	rdd.reduce(lambda x,y : x+y)

	RDD and return a single result	
foreach()	Executes a function on each element of the RDD	rdd.foreach(lambda x: print(x))
saveAsTextFile	Executes the computation and writes the RDD data	rdd.saveAsTextFile("output/path")